

Reporte Técnico Extendido: Complicaciones y Soluciones en la Instalación de Pinguino IDE v12

Documentación detallada de depuración de entorno, gestión de dependencias y reconfiguración de rutas para sistemas basados en PIC32.

Entorno: Plataforma Meta: PIC32MX220F032D-I/
Windows 11 x64 PT (32 bits)

Subsistema de Control: Python 3.7.9 via pyenv

Fecha: Junio
2026 **Estado:** Despliegue Exitoso

Documento: Bitácora de Integración de Software

1. Introducción y Contexto del Sistema

El presente documento constituye un reporte técnico extendido que recopila de manera detallada las complicaciones presentadas, analizadas y resueltas durante el proceso de instalación e integración del entorno de desarrollo modular **Pinguino IDE v12**. Este entorno fue desplegado sobre un sistema operativo Windows de 64 bits utilizando un entorno de ejecución controlado de Python provisto mediante el gestor de versiones `pyenv-win`.

El propósito fundamental de esta instalación es establecer una cadena de herramientas (toolchain) local y estable que permita la compilación, depuración y flasheo de firmware especializado para microcontroladores de arquitectura de 32 bits, centrándose específicamente en el integrado **PIC32MX220F032D-I/PT**. A lo largo del despliegue, surgieron diversas anomalías críticas asociadas a la resolución de dependencias binarias, anidamiento de directorios en el sistema de archivos y discrepancias de direccionamiento absoluto dentro de los archivos de configuración predeterminados del desarrollador original. A continuación, se desglosan de forma exhaustiva cada uno de estos incidentes junto con sus respectivos análisis de causa raíz y protocolos de resolución aplicados.

2. Desglose Exhaustivo de Complicaciones y Soluciones

Caso 1: Error de Localización del Script de Arranque (Falta de Archivo o Directorio)

Al intentar realizar la ejecución inicial de la interfaz gráfica a través de la consola avanzada de PowerShell, el intérprete de Python devolvió un fallo crítico que impidió la inicialización del flujo del programa.

[SÍNTOMA / ERROR EN CONSOLA]

```
PS C:\Pinguino\v12\pinguino-ide> python pinguino-ide.py
python: can't open file 'pinguino-ide.py': [Errno 2] No such file or directory
```

Análisis de Causa Raíz:

El error de entrada/salida [Errno 2] ocurre de manera estricta cuando el motor de ejecución de Python solicita el descriptor de un archivo al sistema operativo y este devuelve un puntero nulo debido a la inexistencia del archivo en la ruta de trabajo actual (Working Directory). Al inspeccionar de forma profunda la estructura física del almacenamiento tras la extracción del paquete zip comprimido, se identificó un problema de **anidamiento redundante de directorios** generado automáticamente por el descompresor. La ruta física real del script ejecutable se localizaba en un subnivel homónimo: `C:\Pinguino\v12\pinguino-ide\pinguino-ide\pinguino-ide.py`, mientras que la terminal se encontraba operando en el nodo padre directo.

[RESOLUCIÓN APLICADA]

1. Se procedió a inspeccionar el contenido del directorio mediante el comando de listado nativo `ls` o `dir` en PowerShell, validando la presencia de la subcarpeta hija.
2. Se corrigió el direccionamiento de la terminal de PowerShell para descender al nivel exacto donde residen los recursos gráficos e internos del programa ejecutable mediante el comando de cambio de directorio:

```
cd pinguino-ide
```

3. Una vez actualizado el prompt a `PS C:\Pinguino\v12\pinguino-ide\pinguino-ide>`, se procedió a re-ejecutar con éxito el comando base de llamada al intérprete:

```
python pinguino-ide.py
```

Caso 2: Conflictos de Direccionamiento Absoluto Extranjero en Archivo de Configuración

Durante la revisión estructural estática de los archivos que gobiernan el comportamiento del IDE, se detectó que el archivo maestro de configuración para plataformas Windows contenía directivas de compilación y llamadas a librerías apuntando a volúmenes y rutas de usuario completamente inexistentes en el sistema local.

[SÍNTOMA / ESTRUCTURA ANÓMALA DETECTADA]

El archivo `pinguino.windows.conf` ubicado en la ruta de configuración gráfica poseía las siguientes líneas nativas del desarrollador original:

```
[Paths]
pinguino_8_libs = D:\Users\Regis\Dvpt\pinguino\pinguino-libraries\p8\
pinguino_32_libs = D:\Users\Regis\Dvpt\pinguino\pinguino-libraries\p32\
install_path = D:\Users\Regis\Dvpt\pinguino
user_path = D:\Users\Regis\Documents\Pinguino\v13
```

Análisis de Causa Raíz:

El software Pinguino IDE v12 almacena las variables de entorno de las herramientas de compilación cruzada de forma estática dentro de archivos con formato INI (`.conf`). Al venir preconfigurado con las rutas absolutas de la máquina de desarrollo original (pertenecientes al volumen `D:` y usuario `Regis`), cualquier intento de compilación de un sketch de C/C++ generaría fallos catastróficos inmediatos (*Toolchain Linker/Compiler Error*), ya que el IDE buscaría de forma persistente los scripts de enlazado y archivos de cabecera (`.h`) en ubicaciones virtuales inaccesibles.

[RESOLUCIÓN APLICADA]

Se realizó una intervención directa sobre el archivo de inicialización, abstrayendo las rutas extranjeras y reemplazándolas por un mapa estricto alineado a la estructura local en el disco principal `C:` , tomando en consideración la doble carpeta descubierta en el Caso 1. El bloque `[Paths]` se reescribió por completo de la siguiente manera:

```
[Paths]
sdcc_bin =
gcc_bin = C:\Pinguino\v12\compilers\p32\bin\
xc8_bin =
pinguino_8_libs =
pinguino_32_libs = C:\Pinguino\v12\pinguino-libraries\p32\
install_path = C:\Pinguino\v12\pinguino-ide\pinguino-ide
user_path = C:\Pinguino\v12\Proyectos
user_libs = C:\Pinguino\v12\Proyectos\pinguinolibs
```

Efecto de la corrección: Se garantizó que el compilador GCC para la arquitectura PIC32 (`mips-elf-gcc`) y las librerías base de bajo nivel específicas para controladores de 32 bits apunten a los directorios físicos locales correctos.

Caso 3: Excepción por Ausencia de Módulo de Terceros (Falta de la Librería Requests)

Tras mitigar las fallas de directorios y mapeo de configuraciones, se procedió a lanzar el hilo principal de ejecución de la interfaz gráfica. Sin embargo, el intérprete interrumpió el proceso arrojando un volcado de pila (Traceback) debido a una dependencia de software no satisfecha a nivel global en el entorno aislado de Python.

[SÍNTOMA / EXCEPCIÓN DE PYTHON]

```
Traceback (most recent call last):
  File "pinguino-ide.py", line 21, in <module>
    from pinguino import pinguino
    ...
  File "C:\Pinguino\v12\pinguino-ide\pinguino-ide\pinguino\qtgui\ide\child_windows\
    import requests
ModuleNotFoundError: No module named 'requests'
```

Análisis de Causa Raíz:

El script secundario `submit_bug.py`, integrado de forma nativa en los submódulos de la interfaz gráfica (QTGUI) de Pinguino para automatizar el reporte de fallas hacia los servidores del proyecto, requiere el módulo externo `requests` para gestionar peticiones HTTP/HTTPS. Aunque la guía de instalación base contemplaba la inyección de librerías esenciales de renderizado (`PySide2`) y comunicación por hardware (`pyusb`), omitía por completo esta dependencia de red de alto nivel dentro del gestor de paquetes `pip` para la versión específica Python 3.7.9.

[RESOLUCIÓN APLICADA]

Se aprovechó que la terminal se encontraba operando bajo el contexto y la variable de entorno correcta de la versión de Python administrada por pyenv, y se procedió a inyectar la librería de forma directa ejecutando el gestor de paquetes:

```
pip install requests
```

El instalador descargó y compiló el paquete `requests` junto con sus dependencias requeridas de transporte seguro (`urllib3` , `idna` , `charset-normalizer` , `certifi`). Inmediatamente posterior a la instalación exitosa, se relanzó el comando de arranque, logrando la inicialización completa del entorno gráfico sin más interrupciones.

3. Protocolo de Inicialización Estándar y Buenas Prácticas

Con la finalidad de evitar la recurrencia de los fallos de entorno analizados y asegurar un espacio de trabajo óptimo para el desarrollo de sistemas embebidos de 32 bits, se recomienda adoptar el siguiente protocolo de arranque secuencial:

[FLUJO DE TRABAJO DIARIO]

1. **Apertura de Consola:** Ejecutar PowerShell en un directorio neutral o directamente apuntando al nodo raíz del proyecto.
2. **Navegación Controlada:** Asegurar el posicionamiento en el nodo ejecutable final mediante: `cd C:\Pinguino\v12\pinguino-ide\pinguino-ide`
3. **Verificación de Python:** Validar que pyenv mantenga fijada la versión 3.7.9 escribiendo `python --version` antes del lanzamiento inicial.
4. **Arranque del IDE:** Lanzar la aplicación con la sintaxis validada: `python pinguino-ide.py` .
5. **Gestión de Proyectos:** Almacenar todos los esquemas de código e implementaciones físicas estrictamente bajo la ruta local creada (`C:\Pinguino\v12\Proyectos\`), mitigando cualquier interferencia de sincronización con nubes comerciales o permisos restrictivos del sistema operativo en las carpetas nativas de usuario.